CO2-AT2

NAME :B. Harsha Vardhan

Reg_no :192425201

SUB CODE:MLA0302

SUB NAME: Reinforcement Learning

Exp 1 :Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality.

CODE :

```
import random
# Ride-sharing RL Simulation
locations = ["Airport", "Mall", "Station", "Office"]
drivers = {
   "D1": "Airport",
   "D2": "Mall",
   "D3": "Station"
}
total_reward = 0
print("===== Ride-Sharing Reinforcement Learning =====\n")
for ride in range(10):
   print("Ride Request", ride + 1)
   passenger = random.choice(locations)
   print("Passenger Location:", passenger)
   print("Driver Locations:")
   for d in drivers:
      print(d, "->", drivers[d])
actions = list(drivers.keys())
action = random.choice(actions)
print("Selected Driver:", action)
```

```
if drivers[action] == passenger:
        waiting_time = random.randint(1, 3)
        reward = 10
        print("Quick Pickup")
    else:
        waiting_time = random.randint(4, 8)
        reward = -5
        print("Late Pickup")
total_reward += reward
print("Waiting Time:", waiting_time, "minutes")
    print("Reward:", reward)
    print("Total Reward:", total_reward)
drivers[action] = random.choice(locations)
print("Driver New Location:", drivers[action])
    print("-----------------------------------")
print("\nSimulation Finished")
print("Final Total Reward:", total_reward)
```

OUTPUT :

```
Ride Request 8
Passenger Location: Airport
Driver Locations:
D1 -> Station
D2 -> Office
D3 -> Office
Selected Driver: D3
Late Pickup
Waiting Time: 6 minutes
Reward: -5
Total Reward: -25
Driver New Location: Airport
-----------------------------------
Ride Request 9
Passenger Location: Office
Driver Locations:
D1 -> Station
D2 -> Office
D3 -> Airport
Selected Driver: D3
Late Pickup
Waiting Time: 6 minutes
Reward: -5
Total Reward: -30
Driver New Location: Mall
-----------------------------------
Ride Request 10
Passenger Location: Station
Driver Locations:
D1 -> Station
D2 -> Office
D3 -> Mall
Selected Driver: D2
Late Pickup
Waiting Time: 5 minutes
Reward: -5
Total Reward: -35
Driver New Location: Mall
-----------------------------------

Simulation Finished
Final Total Reward: -35
```

EXP 2: An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations.

CODE :

```
import random
# Study materials
materials = ["Python", "Java", "SQL", "Machine Learning", "Data Structures"]
# Initial scores for each material
scores = {
    "Python": 0,
    "Java": 0,
    "SQL": 0,
    "Machine Learning": 0,
    "Data Structures": 0
}
epsilon = 0.3   # Exploration probability (30%)
print("===== E-Learning Recommendation using Reinforcement Learning =====")
for i in range(10):
    print("\nRecommendation", i + 1)
    # Exploration or Exploitation
    if random.random() < epsilon:
        material = random.choice(materials)
        print("Strategy: Exploration")
    else:
        material = max(scores, key=scores.get)
        print("Strategy: Exploitation")
    print("Recommended Material:", material)
    # Simulate student feedback
    feedback = random.choice(["Liked", "Not Liked"])
```

```
    print("Student Feedback:", feedback)

    # Reward Function
    if feedback == "Liked":
        reward = 10
    else:
        reward = -5
    scores[material] += reward
    print("Reward:", reward)
    print("Updated Scores:", scores)
print("\n===== Final Scores =====")
for material, score in scores.items():
    print(material, ":", score)
best = max(scores, key=scores.get)
print("\nBest Recommended Material:", best)
```

OUTPUT :

```
Recommendation 6
Strategy: Exploitation
Recommended Material: Java
Student Feedback: Not Liked
Reward: -5
Updated Scores: {'Python': -5, 'Java': 10, 'SQL': -5, 'Machine Learning': 0, 'Data Structures': 0}

Recommendation 7
Strategy: Exploration
Recommended Material: Python
Student Feedback: Liked
Reward: 10
Updated Scores: {'Python': 5, 'Java': 10, 'SQL': -5, 'Machine Learning': 0, 'Data Structures': 0}

Recommendation 8
Strategy: Exploitation
Recommended Material: Java
Student Feedback: Liked
Reward: 10
Updated Scores: {'Python': 5, 'Java': 20, 'SQL': -5, 'Machine Learning': 0, 'Data Structures': 0}

Recommendation 9
Strategy: Exploitation
Recommended Material: Java
Student Feedback: Liked
Reward: 10
Updated Scores: {'Python': 5, 'Java': 30, 'SQL': -5, 'Machine Learning': 0, 'Data Structures': 0}

Recommendation 10
Strategy: Exploration
Recommended Material: Machine Learning
Student Feedback: Liked
Reward: 10
Updated Scores: {'Python': 5, 'Java': 30, 'SQL': -5, 'Machine Learning': 10, 'Data Structures': 0}

===== Final Scores =====
Python : 5
Java : 30
SQL : -5
Machine Learning : 10
Data Structures : 0

Best Recommended Material: Java
```

EXP 3 : Design and develop a Reinforcement Learning framework a robotic vacuum cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time.

CODE :

```
import random
# 5x5 Room
GRID_SIZE = 5
# Obstacles
obstacles = [(1, 2), (3, 3)]
# Dirty cells
dirt = [(0, 1), (2, 2), (4, 4), (1, 4)]
# Robot starting position
robot = [0, 0]
# Value function
value = {}
# Possible actions
actions = ["UP", "DOWN", "LEFT", "RIGHT"]
total_reward = 0
print("===== Robotic Vacuum Cleaner using Reinforcement Learning =====")
for step in range(15):

    print("\nStep", step + 1)
    print("Current Position:", tuple(robot))
# Policy: Randomly choose an action
    action = random.choice(actions)
    print("Action:", action)
new_row = robot[0]
    new_col = robot[1]
```

```
if action == "UP":
        new_row -= 1
    elif action == "DOWN":
        new_row += 1
    elif action == "LEFT":
        new_col -= 1
    elif action == "RIGHT":
        new_col += 1
# Check wall
    if new_row < 0 or new_row >= GRID_SIZE or new_col < 0 or new_col >= GRID_SIZE:
        reward = -5
        print("Hit Wall")
    elif (new_row, new_col) in obstacles:
        reward = -10
        print("Hit Obstacle")
    else:
        robot = [new_row, new_col]
        if tuple(robot) in dirt:
            reward = 10
            dirt.remove(tuple(robot))
            print("Cleaned Dirt")
        else:
            reward = -1
            print("Empty Cell")
    total_reward += reward
    # Update Value Function
    state = tuple(robot)
    if state not in value:
        value[state] = 0
    value[state] += reward
    print("Reward:", reward)
    print("Total Reward:", total_reward)
```

```
    print("State Value:", value[state])
print("\n===== Simulation Completed =====")
print("Final Total Reward:", total_reward)
print("\nValue Function")
for state, val in value.items():
    print(state, ":", val)
```

OUTPUT :

```
Step 12
Current Position: (4, 2)
Action: DOWN
Hit Wall
Reward: -5
Total Reward: 2
State Value: -6

Step 13
Current Position: (4, 2)
Action: UP
Empty Cell
Reward: -1
Total Reward: 1
State Value: -2

Step 14
Current Position: (3, 2)
Action: DOWN
Empty Cell
Reward: -1
Total Reward: 0
State Value: -7

Step 15
Current Position: (4, 2)
Action: DOWN
Hit Wall
Reward: -5
Total Reward: -5
State Value: -12

===== Simulation Completed =====
Final Total Reward: -5

Value Function
(0, 1) : 10
(0, 0) : -6
(1, 0) : -1
(1, 1) : -1
(2, 1) : -2
(2, 2) : 10
(3, 1) : -1
(3, 2) : -2
(4, 2) : -12
```

EXP 4 : A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance.

CODE :

import random

```
# Cryptocurrency prices
prices = [100, 102, 101, 105, 103, 108, 106, 110, 107, 112]
actions = ["BUY", "SELL", "HOLD"]
balance = 1000
crypto = 0
total_reward = 0
print("===== Cryptocurrency Trading Reinforcement Learning =====")
for day in range(len(prices)-1):
    print("\nDay:", day + 1)
    current_price = prices[day]
    next_price = prices[day + 1]
    print("Current Price: $", current_price)
    # State
    if next_price > current_price:
        state = "Price Increasing"
    else:
        state = "Price Decreasing"
    print("State:", state)
    # Agent Action
    action = random.choice(actions)
    print("Action:", action)
    # Reward Function
    if action == "BUY":
        if next_price > current_price:
            reward = 10
            crypto += 1
            balance -= current_price
        else:
            reward = -10
    elif action == "SELL":
        if crypto > 0 and next_price < current_price:
            reward = 10
```

```
            crypto -= 1
            balance += current_price
        else:
            reward = -5
    else:  # HOLD
        reward = 2 if abs(next_price-current_price) <= 2 else -1
    total_reward += reward
    print("Reward:", reward)
    print("Balance:", balance)
    print("Crypto Owned:", crypto)
    print("Total Reward:", total_reward)
print("\n===== Trading Completed =====")
print("Final Balance:", balance)
print("Final Crypto:", crypto)
print("Total Reward:", total_reward)\
```

OUTPUT :

```
Day: 6
Current Price: $ 108
State: Price Decreasing
Action: SELL
Reward: -5
Balance: 1000
Crypto Owned: 0
Total Reward: -12

Day: 7
Current Price: $ 106
State: Price Increasing
Action: HOLD
Reward: -1
Balance: 1000
Crypto Owned: 0
Total Reward: -13

Day: 8
Current Price: $ 110
State: Price Decreasing
Action: BUY
Reward: -10
Balance: 1000
Crypto Owned: 0
Total Reward: -23

Day: 9
Current Price: $ 107
State: Price Increasing
Action: HOLD
Reward: -1
Balance: 1000
Crypto Owned: 0
Total Reward: -24

===== Trading Completed =====
Final Balance: 1000
Final Crypto: 0
Total Reward: -24
```

EXP 5 : Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism.

CODE :

```
import random
```

```
# Smart Grid Reinforcement Learning Simulation
actions = ["Increase Supply", "Decrease Supply", "Maintain Supply"]
# Initial values
power_supply = 50
total_reward = 0
print("===== Smart Grid Reinforcement Learning System =====")
for hour in range(1, 11):
    print("\nHour:", hour)
    # State: Electricity demand
    demand = random.randint(30, 80)
    print("Electricity Demand:", demand)
    print("Current Power Supply:", power_supply)
    # Select action (Policy)
    action = random.choice(actions)
    print("Action:", action)
    # Environment response
    if action == "Increase Supply":
        power_supply += 10
    elif action == "Decrease Supply":
        power_supply -= 10
    else:
        power_supply = power_supply
    # Reward Function
    difference = abs(power_supply - demand)
    if difference <= 5:
        reward = 10
        print("Status: Balanced Power Distribution")
    elif power_supply < demand:
        reward = -10
        print("Status: Power Shortage")
    else:
        reward = -5
```

```
        print("Status: Power Wastage")
    total_reward += reward
    print("New Power Supply:", power_supply)
    print("Reward:", reward)
    print("Total Reward:", total_reward)
print("\n===== Simulation Completed =====")
print("Final Total Reward:", total_reward)
```

OUTPUT :

```
Hour: 7
Electricity Demand: 41
Current Power Supply: 40
Action: Increase Supply
Status: Power Wastage
New Power Supply: 50
Reward: -5
Total Reward: -5

Hour: 8
Electricity Demand: 68
Current Power Supply: 50
Action: Maintain Supply
Status: Power Shortage
New Power Supply: 50
Reward: -10
Total Reward: -15

Hour: 9
Electricity Demand: 34
Current Power Supply: 50
Action: Increase Supply
Status: Power Wastage
New Power Supply: 60
Reward: -5
Total Reward: -20

Hour: 10
Electricity Demand: 64
Current Power Supply: 60
Action: Increase Supply
Status: Power Wastage
New Power Supply: 70
Reward: -5
Total Reward: -25

===== Simulation Completed =====
Final Total Reward: -25
```

6. A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes.

CODE:

```
import random
states = ["Low Health", "Stable Health", "Critical Health"]
actions = ["Increase Dosage", "Maintain Dosage", "Decrease Dosage"]
total_reward = 0
print("Healthcare Monitoring using Reinforcement Learning\n")
for episode in range(10):
    state = random.choice(states)
    action = random.choice(actions)
    print("Episode:", episode + 1)
    print("State:", state)
    print("Action:", action)
    if state == "Critical Health" and action == "Increase Dosage":
        reward = 10
    elif state == "Stable Health" and action == "Maintain Dosage":
        reward = 8
    elif state == "Low Health" and action == "Decrease Dosage":
        reward = 6
    else:
        reward = -5
    total_reward += reward
    print("Reward:", reward)
    print()
print("Training Completed")
print("Total Reward:", total_reward)
```

OUTPUT:

7. Design a model drone delivery system uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization.

CODE:

```
import random
states = ["Open Path", "Traffic Area", "Obstacle Ahead"]
actions = ["Move Forward", "Take Alternate Route", "Hover"]
total_reward = 0
print("Drone Delivery using Reinforcement Learning\n")
for episode in range(10):
    state = random.choice(states)
    action = random.choice(actions)
    print("Episode:", episode + 1)
    print("State:", state)
    print("Action:", action)
    if state == "Open Path" and action == "Move Forward":
        reward = 10
    elif state == "Traffic Area" and action == "Take Alternate Route":
        reward = 8
    elif state == "Obstacle Ahead" and action == "Hover":
        reward = 6
    else:
```

```
        reward = -5
    total_reward += reward
    print("Reward:", reward)
    print()
print("Training Completed")
print("Total Reward:", total_reward)
```

OUTPUT:

```
Episode: 5
State: Obstacle Ahead
Action: Move Forward
Reward: -5

Episode: 6
State: Open Path
Action: Take Alternate Route
Reward: -5

Episode: 7
State: Obstacle Ahead
Action: Hover
Reward: 6

Episode: 8
State: Obstacle Ahead
Action: Move Forward
Reward: -5

Episode: 9
State: Obstacle Ahead
Action: Move Forward
Reward: -5

Episode: 10
State: Obstacle Ahead
Action: Hover
Reward: 6

Training Completed
Total Reward: -28
```

8. Design and develop a Reinforcement Learning framework a manufacturing robot uses Reinforcement Learning to optimize assembly operations. Apply the concept of policy and explain how value functions help improve efficiency.

CODE:

```
import random
states = ["Parts Ready", "Assembly Running", "Assembly Error"]
actions = ["Assemble Part", "Continue Assembly", "Repair Machine"]
total_reward = 0
print("Manufacturing Robot using Reinforcement Learning\n")
for episode in range(10):
    state = random.choice(states)
    action = random.choice(actions)
    print("Episode:", episode + 1)
    print("State:", state)
    print("Action:", action)
```

```
    if state == "Parts Ready" and action == "Assemble Part":
        reward = 10
    elif state == "Assembly Running" and action == "Continue Assembly":
        reward = 8
    elif state == "Assembly Error" and action == "Repair Machine":
        reward = 6
    else:
        reward = -5
    total_reward += reward
    print("Reward:", reward)
    print()
print("Training Completed")
print("Total Reward:", total_reward)
```

OUTPUT:

9. An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance.

CODE:

```
import random
states = ["Low Click Rate", "Medium Click Rate", "High Click Rate"]
actions = ["Show New Ad", "Keep Current Ad", "Show Premium Ad"]
total_reward = 0
```

```
print("Online Advertising using Reinforcement Learning\n")
for episode in range(10):
    state = random.choice(states)
    action = random.choice(actions)
    print("Episode:", episode + 1)
    print("State:", state)
    print("Action:", action)
    if state == "High Click Rate" and action == "Show Premium Ad":
        reward = 10
    elif state == "Medium Click Rate" and action == "Keep Current Ad":
        reward = 8
    elif state == "Low Click Rate" and action == "Show New Ad":
        reward = 6
    else:
        reward = -5
    total_reward += reward
    print("Reward:", reward)
    print()
print("Training Completed")
print("Total Reward:", total_reward)
```

OUTPUT:

10. A smart irrigation system uses Reinforcement Learning to optimize water usage. Create an RL framework by defining states, actions, and reward function.

CODE:

```
import random
states = ["Dry Soil", "Normal Soil", "Wet Soil"]
actions = ["Increase Water", "Maintain Water", "Stop Watering"]
total_reward = 0
print("Smart Irrigation using Reinforcement Learning\n")
for episode in range(10):
    state = random.choice(states)
    action = random.choice(actions)
    print("Episode:", episode + 1)
    print("State:", state)
    print("Action:", action)
    if state == "Dry Soil" and action == "Increase Water":
        reward = 10
    elif state == "Normal Soil" and action == "Maintain Water":
        reward = 8
    elif state == "Wet Soil" and action == "Stop Watering":
        reward = 6
    else:
        reward = -5
    total_reward += reward
    print("Reward:", reward)
    print()
print("Training Completed")
print("Total Reward:", total_reward)
```

OUTPUT:

```
Episode: 5
State: Dry Soil
Action: Stop Watering
Reward: -5

Episode: 6
State: Dry Soil
Action: Maintain Water
Reward: -5

Episode: 7
State: Wet Soil
Action: Increase Water
Reward: -5

Episode: 8
State: Dry Soil
Action: Maintain Water
Reward: -5

Episode: 9
State: Dry Soil
Action: Stop Watering
Reward: -5

Episode: 10
State: Wet Soil
Action: Stop Watering
Reward: 6

Training Completed
Total Reward: -28
>>>
```